

DepChain Report

Pedro Curto¹, Pedro Ribeiro¹, and Rodrigo Ganância¹

Instituto Superior Técnico, Portugal
pedro.a.curto@tecnico.ulisboa.pt
rodrigo.ganancia@tecnico.ulisboa.pt
pedro.melo.ribeiro@tecnico.ulisboa.pt

Project Introduction

Goals

The project aims to develop a simplified permissioned blockchain system with high dependability guarantees (DepChain).

The initial delivery focuses on establishing secure and reliable communication between blockchain members and clients and implements a Consensus protocol to ensure consistent and dependable state updates across all members of the system.

System Assumptions

The system operates under the assumption of a static membership and a pre-established Public Key Infrastructure. The application can trust its local instance of the client library.

The blockchain operates on an unreliable and unsecured network, requiring the system to handle dropped and corrupted packets.

Additionally, some members may be malicious and act arbitrarily. While the leader is assumed to be fixed and correct, the algorithms must be designed to tolerate potential Byzantine behavior (leader included) and allow for future extensions.

Our system assumes that N , the number of members, is $3f + 1$, with f being the number of faults it tolerates.

Design Requirements

The implementation must always guarantee safety and ensure liveness only when the leader is correct. The system should remain functional as long as the number of Byzantine processes does not exceed $(n + f) / 2$.

A Byzantine process must not be able to add a value to the blockchain that was not submitted by a client, lead to inconsistencies where two correct processes maintain different versions of the blockchain, or, as long as the leader is correct, cause the system to stall.

Implementation

Authenticated Perfect Links abstraction

Sessions

At startup, each blockchain member establishes a secure session with every other member, while clients initiate a session with their designated member for communication.

To streamline connections, each member is responsible for initiating the protocol with members that have higher port numbers. Similarly, clients must initiate the protocol with the member to which they will send requests.

The initiating entity generates a new AES key to be used as the session key and encrypts it using the recipient's public key. It then signs the message and transmits it to the other member.

Upon receiving the message, the recipient verifies the signature and decrypts the session key. Once established, all subsequent messages exchanged between these members must include an HMAC generated with the session key to ensure message integrity.

Reliable Transmission

Each entity maintains a sequence number for both sending and receiving messages during a session.

When sending a message, the sender appends its current sequence number to the message and waits for an acknowledgment (ACK) with the same number. If no ACK is received, the message is retransmitted at a fixed rate until the ACK is received, confirming successful delivery. Once the ACK is received, the sender increments its sequence number and notifies the application of successful message delivery.

Upon receiving a message, the receiver checks if the message's sequence number matches its expected receiving counter. If they match, the receiver sends an ACK and delivers the message to the application. If the message number is lower, it indicates the message has already been delivered, and the receiver only sends the ACK.

During connection establishment, both entities synchronize their sequence numbers to ensure consistency and a message is only accepted, even having the correct sequence number, if the HMAC is verified successfully.

Client library

In this initial delivery, the client library includes only the APPEND command. The client is prompted to provide a string that it wishes to append to the blockchain and receives a response indicating whether the operation was successful, along with the epoch instance in which the value was appended.

When sending the append request, the client signs the value with its private key and sends it to the member's leader, guaranteeing the authenticity of the value when added to the chain. If the value has not been appended for an extended period, the member may try to forward it to another member. However,

since leader changes are not yet implemented, in this stage of the project, the message should be sent to the leader to be proposed.

Byzantine Read/Write Epoch Consensus

A Byzantine Quorum corresponds to $N - f$, which is $2f + 1$ processes.

Upon receiving an APPEND message, the leader starts a new Consensus instance by proposing the value received. If multiple append messages are received while a Consensus instance is incurring, they accumulate on a queue waiting for their turn to be proposed onto the blockchain.

The Consensus algorithm follows the Byzantine Read/Write Epoch Consensus algorithm from the course book [1] (Algorithms 5.17 and 5.18).

Similarly to the Algorithm from the book, it consists of two main phases, the **Read** and the **Write** phases.

Read Phase:

- The leader starts by setting its $\langle value, ts \rangle$ pair to the value present in the client's **APPEND**, and the timestamp to zero. Afterwards, it broadcasts a **READ** to all members.
- The leader always waits for a specified **STATE_TIMEOUT**, to allow all correct processes to join in. We will improve this in the next delivery to leave as soon as they received a byzantine quorum of messages, where a value that can be decided is present. During this time, each member receives the leader's **READ** request and sends its local **STATE** message, consisting of its $\langle value, ts \rangle$ pair, writeset, epoch, instance of the consensus, and signature over these contents. The instance of consensus is needed to prevent valid messages from previous instances to enter on following decisions, and the signature guarantees that when the leader sends the list of the states, the member can confirm the authenticity of each one.
- The leader collects all of these **STATE** messages and broadcasts them to all members, in the form of a **COLLECTED** message. At this point, all members, including the leader, will move to the *decideValue* function, that will decide what value each member chooses to write on the **WRITE** phase.
- After receiving the **COLLECTED** message, all processes ensure that they received at least a quorum of messages and check the signature of every **STATE** message inside it. Members select a value by choosing the $\langle value, ts \rangle$ pair with highest timestamp that appears in at least $N - f$ write sets. When counting the number of times in which it appears on the write set, the timestamp of the selected pair must be less than or equal to those of the same value present in the write sets. If no such pair exists, the default value is taken from the leader.

Write Phase:

- Each process broadcasts in a **WRITE** message, the value that it decided. It must wait for $N - f$ **WRITES** of the same value to **DECIDE**. Each process

only keeps a WRITE message from each process. Upon receiving the quorum of WRITES it broadcasts a DECIDE message. It then waits for $N - f$ DECIDE messages for the same value and, after, it delivers it to the blockchain and the Consensus instance ends. Every time a process is waiting for a quorum of messages, it timeouts after a period of time, aborting if it does not reach the quorum needed to advance, and moving on to the next epoch.

Tests

All of the automatic tests (JUnit) that we will describe are present within the *member* package, in two distinct classes.

There is also the possibility of running your own manual tests, using the appropriate shell scripts to launch either all members as non Byzantine, or a Byzantine member with configurable behaviour - the way of accomplishing any of this or the tests below is described in our *README.md*.

A worthy note is that our system **isn't currently able of handling certain special characters** (e.g., ^o, à, á, ..), relating to our use of *gson*. We will fix this in the next delivery.

Regular Scenario

A simple test was included to show the behaviour of the program with a simple sequence of appends sent from the client to members that are all non-byzantine.

Predicting Network Unreliability

There can be various types of message faults in the network namely:

- **Loss:** Messages may be dropped.
- **Duplication:** Messages may be sent multiple times.
- **Delays:** Messages may experience unpredictable delays.
- **Corruption:** Messages may have replaced, inserted, or deleted characters.

For testing network unreliability, a perfect link abstraction class **PerfectLinkByzantine** was implemented. This class potentially creates any of these faults at any moment. Currently, we were able to implement a test within the *ByzantineBehaviourTest* class that simulates all of the members potentially experiencing delays in the network, and will expand to the remaining faults in the next delivery.

Simulating Byzantine Behaviour

To cope with possible Byzantine behaviour from members, we devised a series of scenarios that corroborate the correct functioning of the program with these adversarial conditions. We devised a replay scenario where a byzantine process

collects a correctly signed message from another instance and tries to propose it in a later instance, but it was not finished in time for this delivery deadline.

Ignoring Messages: A subset of f processes ignore all messages, essentially becoming useless members. Since the system tolerates this failure, it continues appending incoming messages to the Blockchain, and the system isn't disrupted.

Spam Messages: A subset of f processes spam the leader with a configurable number of messages - these include `STATE`, `WRITE` and `ACCEPT` messages. The service continues functioning normally.

Impersonation: The byzantine process picks another member at random, and attempts to send the message as if it was him. The test asserts that, despite this attempt, the leader notices that an invalid signature was detected, doesn't consider that `STATE`, and the algorithm decides a correct value.

Incorrect States: The byzantine processes try to propose a value to the consensus with a higher timestamp than it was supposed to, exploiting the fact that the other correct processes would be tempted to pick it and then send broadcast `WRITE` and `ACCEPT` messages for it. However, since $n - f$ did not propose this value, this attempt fails and the system continues working correctly.

Wrong Writes and Accepts: Finally, a subset of byzantine processes send incorrect `WRITE` and `ACCEPT` messages, but fail to disrupt the system.

Conclusion

Although our implementation does not include leader changes, it ensures that the leader cannot introduce arbitrary values into the blockchain. The consensus algorithm prevents the break of consistency when Byzantine processes sending arbitrarily wrong messages to the protocol and when a value gets decided it is added to every member's blockchain.

Under a correct leader, the protocol ensures that sufficient responses are eventually gathered during the `READ` phase, a Byzantine Quorum is formed in the `WRITE` phase, and instances are finalized with a Byzantine Quorum of `ACCEPT` messages.

References

1. Christian Cachin, Rachid Guerraoui, and Luís Rodrigues. *Introduction to Reliable and Secure Distributed Programming*. 2nd Ed. Springer, 2011.